

Decoupled Image-to-Video Generation via Latent Flow: An Investigation into Embedding-Conditioned Rendering

ZhuFeiHao 20949355

The Hong Kong University of Science and Technology

fhzhu@connect.ust.hk

https://github.com/yugeng58/COMP4471_PROJECT

Abstract

Image-to-video generation from a single static image and an action label requires simultaneously understanding both what is in the scene and how it should evolve over time. We investigate a decoupled framework inspired by the Joint-Embedding Predictive Architecture (JEPA): a causal Transformer (Flow Model) predicts the trajectory of CLIP embeddings in a compact 512-dimensional semantic space, and a conditioned UNet (Renderer) converts each predicted embedding back to a pixel frame. We conduct a systematic set of experiments across four Renderer architectures—FiLM conditioning, cross-attention to CLIP patch tokens, contrastive conditioning loss, and optical flow prediction—and find that each variant collapses to a mode where the embedding condition is effectively ignored and the output is a near-identity transformation of the input frame. We present a detailed failure analysis identifying the primary root cause: frozen CLIP ViT embeddings, trained for semantic image-text alignment, lack the spatial configuration and dense correspondence information necessary for pixel-level frame rendering. Contributing factors—loss functions that reward near-identity outputs and conditioning mechanisms unable to express spatial motion—are understood as consequences of this information bottleneck. Despite the negative result, our analysis clarifies the requirements for embedding-conditioned video synthesis and suggests concrete alternative approaches.

1. Introduction

Image-to-video (I2V) generation aims to produce a temporally coherent video sequence from a single static frame. When conditioned on an action label (e.g., “walking”), the generated video should depict natural motion consistent with that action. This task is relevant to content creation, video editing, and synthetic data augmentation for action recognition systems.

Recent large-scale diffusion models such as Stable Video Diffusion and Wan2.2 have demonstrated impressive results by inflating image diffusion architectures into the temporal domain. However, these end-to-end approaches incur substantial computational costs and remain prone to motion flickering and identity drift.

Inspired by the JEPA framework, we investigate an alternative: decoupling the problem into a **Flow Model** that predicts motion in a semantic embedding space and a **Renderer** that converts semantic predictions back to pixels. The key hypothesis is that motion dynamics are easier to model in a compact, abstract embedding space than in raw pixels. In an ideal scenario, the Flow Model would learn high-level action dynamics (“a walking person alternates legs”), and the Renderer would faithfully translate these semantic blueprints into frames.

A central motivation for using frozen pre-trained encoders (CLIP ViT-B/32) was to investigate whether their rich visual-semantic priors could reduce training costs. Large-scale video generation models typically require enormous datasets and compute budgets; if a frozen pre-trained representation already captures meaningful visual structure, a lightweight decoder might be sufficient to render motion within that representation space. This is the core hypothesis we set out to test: **can frozen pre-trained visual priors serve as an effective backbone for low-cost image-to-video generation?**

We implement this framework using frozen CLIP ViT-B/32 encoders, a causal Transformer for embedding trajectory prediction, and a UNet-based Renderer with conditioning injected via FiLM, cross-attention, or explicit flow prediction heads. We systematically ablate the conditioning mechanism and loss function across four Renderer variants.

Our key finding is negative but instructive: across all four variants, the Renderer learns to ignore the embedding condition and produces near-identical outputs regardless of input conditioning. The primary root cause, as our analysis will show, is that frozen CLIP ViT embeddings—trained for semantic alignment rather than spatial correspondence—

lack the information content necessary for pixel-level rendering, a bottleneck that no conditioning architecture or loss function could overcome.

We present a detailed failure analysis quantifying the embedding signal strength, the collapse behavior across architectures, and the training dynamics. We believe these findings provide useful guidance for future work on embedding-conditioned generative models.

2. Related Work

2.1. Image-to-Video Generation

Early video generation methods employed recurrent networks or generative adversarial networks. More recently, diffusion-based approaches have become standard. Stable Video Diffusion fine-tunes a pre-trained image diffusion model on video data by adding temporal layers. Wan2.2 follows a similar approach at larger scale. These models generally operate in a VAE latent space, simultaneously modeling spatial structure and temporal dynamics in one pipeline. We experiment with a fundamentally different architecture that attempts to factor these responsibilities across two modules.

2.2. Latent Predictive Architectures

The JEPA framework proposes that predictive learning should operate in representation space rather than input space. V-JEPA applies this principle to self-supervised video learning by predicting masked spatio-temporal features. Our Flow Model follows the same intuition: predict the trajectory of CLIP embeddings rather than raw pixels. Where our work differs is the downstream rendering step—converting predicted embeddings back to pixels—which proves to be the critical bottleneck.

2.3. Conditioned Image Generation

Feature-wise Linear Modulation (FiLM) and cross-attention are standard techniques for conditioning image generation on external information. FiLM applies channel-wise affine transformations $\gamma(c) \odot x + \beta(c)$ predicted from a conditioning vector c , and has been widely used in visual question answering, style transfer, and video prediction. Cross-attention, popularized by the Transformer architecture, allows each spatial position in a feature map to attend to a set of conditioning tokens. Our experiments test both mechanisms independently and in combination, and find that neither is sufficient when the conditioning signal (adjacent-frame CLIP embeddings) varies by less than 3% in cosine distance.

2.4. Optical Flow and Warping

Frame interpolation and video prediction often employ explicit optical flow prediction followed by warping. Methods

such as RAFT achieve state-of-the-art optical flow estimation through iterative refinement, while simpler approaches use Farneback or TV-L1 flow as a supervision signal. Our final Renderer variant adopts a flow-based approach: a UNet backbone predicts a dense flow field conditioned on CLIP embeddings, and the previous frame is warped via differentiable bilinear sampling. We use Farneback optical flow as pseudo-supervision and apply total variation regularization to encourage spatial smoothness.

2.5. Action-Conditioned Video Generation

Motion generation conditioned on action labels has been studied in controlled settings such as video game environments and simplified human motion benchmarks. Text-to-video models such as Make-A-Video and Imagen Video use large language model embeddings for open-vocabulary conditioning at scale. Our setting is intermediate: we use closed-set action labels encoded through CLIP text embeddings, providing a clean semantic signal in the same embedding space as the visual features.

3. Data

We use the HMDB51 dataset, containing 6,754 video clips across 51 human action categories such as “walking,” “drinking,” “brushing hair,” and “climbing stairs.” Videos average approximately 3 seconds in duration at 25–30 fps. All available videos (6,754) are used for training; metrics reported in this paper are computed on the training set, as our investigation focused on training dynamics and conditioning sensitivity rather than generalization to held-out data.

3.1. Preprocessing

For each video, we sample 17 consecutive frames ($n = 16$ prediction steps) at a stride of 2, and resize all frames to 224×224 pixels. CLIP embeddings (CLS token + 49 patch tokens) are pre-extracted for the entire dataset (242MB) to avoid redundant forward passes during Flow Model training. The Renderer samples random frame pairs (I_{t-1}, I_t) from each video with a gap of 1 frame.

3.2. Embedding Analysis

A key diagnostic we perform early is measuring the variation in CLIP embeddings across frames. Table 1 summarizes pairwise cosine similarities.

The critical observation is that adjacent-frame embeddings differ by only $1 - \cos \approx 0.024$ on average. This means the embedding trajectory that the Flow Model must predict, and that the Renderer must use as a conditioning signal, contains very little per-frame variation. Any learning algorithm must amplify this tiny signal by orders of magnitude to produce meaningful pixel-space changes.

Table 1. CLIP ViT-B/32 embedding cosine similarity. Same-video frames are highly similar (1-cos = 0.024 for adjacent frames), limiting the information available for frame-to-frame motion prediction.

Comparison	Mean cos
Same video, distant frames ($\Delta t \approx 60$)	0.862
Same video, adjacent frames ($\Delta t = 1$)	0.976
Different videos, same action class	0.664
Different action classes	0.564

4. Methods

Our framework consists of three frozen encoders and two trainable modules. The pipeline is as follows: (1) a frozen CLIP ViT encodes the input image I_0 to E_0 and extracts spatial patch tokens; (2) a frozen CLIP text encoder produces the label embedding L ; (3) the Flow Model—a causal Transformer—predicts the embedding trajectory $(\hat{E}_1, \dots, \hat{E}_n)$ from (L, E_0) using shifted teacher forcing; (4) the Renderer—a UNet with FiLM and cross-attention conditioning—produces each frame I_t from I_{t-1} , conditioned on $\hat{E}_t$, L , and patch tokens. The two models are trained separately.

4.1. Encoders

Image Encoder. Frozen CLIP ViT-B/32. We extract both the 512-dim CLS embedding E_t and the 49 spatial patch tokens (7×7 grid, 512-dim each) from the final vision transformer layer.

Label Encoder. Frozen CLIP text encoder. Action labels are converted from directory names to natural text (e.g., “brush hair”) and encoded to 512-dim semantic embeddings L .

4.2. Flow Model

We initially attempted a continuous flow matching formulation: learning a velocity field $v_\theta(E_t, t, L)$ that transports E_0 to E_n through the embedding space. This approach produced unstable trajectories and poor convergence, which motivated our switch to a discrete autoregressive formulation.

Our final Flow Model is a 4-layer causal Transformer encoder (12.9M parameters) with 8 attention heads and feedforward dimension 2048. The input sequence is $[L, E_0, E_1, \dots, E_{n-1}]$, and the model is trained with shifted teacher forcing: position t predicts $\hat{E}_t$ from $(L, E_0, \dots, E_{t-1})$.

We compare two loss functions:

- **MSE:** $\mathcal{L} = \frac{1}{n} \sum_t \|\hat{E}_t - E_t\|_2^2$
- **Cosine:** $\mathcal{L} = \frac{1}{n} \sum_t (1 - \cos(\hat{E}_t, E_t))$

4.3. Renderer

The Renderer is a UNet (4 down/up blocks, 40–43M parameters depending on variant) that takes I_{t-1} as input and produces an output conditioned on $\hat{E}_t$, L , and patch tokens. We experiment with four variants:

1. **FiLM (baseline):** channel-wise Feature-wise Linear Modulation from a condition vector $[\hat{E}_t; L] \in \mathbb{R}^{1024}$, applied at each residual block. Output: pixel residual ΔI_t . Loss: $L1(\Delta I_t, I_t - I_{t-1})$.
2. **FiLM + Cross-Attention:** FiLM conditioning plus multi-head cross-attention from UNet feature maps to CLIP patch tokens at the bottleneck and first two upsampling blocks. Output: pixel residual ΔI_t . Loss: L1.
3. **FiLM + Cross-Attention + Contrastive:** same as variant 2, plus a contrastive conditioning loss: for a batch, we compute $\mathcal{L}_{\text{cont}} = \|\cos(\Delta I_{\text{real}}, \Delta I_{\text{fake}}) - \cos(E_{\text{real}}, E_{\text{fake}})\|_2^2$, where fake conditions are shuffled embeddings from other videos in the batch. This explicitly penalizes the Renderer for producing identical outputs from different conditions.
4. **Flow Prediction:** same UNet backbone with FiLM + cross-attention, but the output head predicts a dense optical flow field $\in \mathbb{R}^{2 \times 224 \times 224}$. The previous frame is warped via the predicted flow: $I_t = \text{warp}(I_{t-1}, \text{flow})$. Loss: $L1(I_t^{\text{warped}}, I_t^{\text{gt}}) + \lambda_{\text{TV}} \cdot \text{TV}(\text{flow}) + \lambda_{\text{FB}} \cdot L1(\text{flow}, \text{Farneback})$.

4.4. Training Details

All models are trained with AdamW ($lr = 10^{-4}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, weight decay 10^{-2}) and a cosine annealing learning rate schedule over 100 epochs. We use a batch size of 8 with gradient accumulation over 2 steps (effective batch size 16). Gradient clipping with maximum norm 1.0 is applied. All experiments were conducted on a single NVIDIA RTX 4060 Laptop GPU (8GB VRAM). Complete training of the Flow Model takes approximately 4 hours; each Renderer variant takes approximately 7 hours.

Flow Model training data pipeline. Because passing CUDA models through PyTorch DataLoader workers is not supported on Windows, we pre-extract CLIP embeddings for all 6,754 videos (17 frames each) and save them to disk as a single 242MB file. The Flow Model training loop loads pre-extracted embeddings directly, requiring no CLIP forward passes during training. This eliminates the DataLoader CUDA issue and accelerates training by approximately $3 \times$.

Renderer training data pipeline. The Renderer uses HMDB51FrameDataset, which loads raw frames from disk (via OpenCV) and performs image transformations in the DataLoader workers. CLIP encoding happens in the main training loop where CUDA is available. Farneback optical flow is computed on CPU using OpenCV, adding approximately 30% overhead per batch.

4.5. Implementation Notes

Shifted teacher forcing. An earlier version of the Flow Model used “zero-placeholder” inputs where the model received $[L, E_0, \mathbf{0}, \dots, \mathbf{0}]$ and was expected to predict all future embeddings in a single forward pass. This created a train-inference mismatch: at inference time, the model sees its own predictions rather than zeros. Shifted teacher forcing, where the model receives the real embedding sequence and predicts one step ahead at each position, aligns training and inference exactly. We found this improved embedding trajectory quality significantly (from 2.0% to 82.5% of GT motion, combined with the cosine loss change).

Multi-scale flow prediction. In the flow prediction Renderer variant, we considered predicting flow at multiple decoder resolutions and combining them via coarse-to-fine refinement (similar to RAFT). However, initial experiments showed that even a single-scale flow head struggled to produce consistent non-zero outputs, so we deferred multi-scale investigation to future work.

5. Experiments

5.1. Evaluation Protocol

We evaluate using the following metrics:

- **Embedding Trajectory MSE:** MSE between predicted and ground-truth CLIP embedding sequences.
- **Inter-frame 1-cos Distance:** $1 - \cos(\hat{E}_t, \hat{E}_{t+1})$, measuring how much “motion” the predicted embeddings contain.
- **Conditioning Sensitivity:** $\frac{\|\text{Renderer}(I, E_t) - \text{Renderer}(I, E_0)\|_1}{\|\text{Renderer}(I, E_0)\|_1 + \epsilon}$, measuring whether changing the embedding changes the Renderer output.

5.2. Flow Model Results

Table 2 summarizes the Flow Model results.

Table 2. Flow Model performance with MSE vs. cosine similarity loss. Cosine loss prevents embedding collapse while maintaining prediction accuracy.

Metric	MSE Loss	Cosine Loss
Train loss (final)	0.0116	0.0264
Inter-frame 1-cos (GT)	0.0290	0.0290
Inter-frame 1-cos (pred)	0.0006	0.0239
% of GT motion preserved	2.0%	82.5%

The MSE-trained Flow Model achieves low MSE (0.0116) but collapses to predicting nearly identical embeddings at every timestep (1-cos $\approx$ 0.0006). Cosine loss resolves this: predicted embeddings retain 82.5% of the ground-truth motion variation while maintaining comparable embedding quality (MSE between predicted and GT embeddings actually improves in terms of cosine distance).

5.3. Renderer Results

Table 3 presents the core negative result: across all four architectures, the Renderer learns to ignore the embedding condition.

Table 3. Renderer conditioning sensitivity across four architecture variants. A ratio of ~ 1.0 indicates the model produces identical outputs regardless of the embedding condition.

Renderer Variant	Sensitivity Ratio
1. FiLM + L1	1.03x
2. FiLM + Cross-Attn + L1	0.98x
3. FiLM + Cross-Attn + L1 + Contrastive	0.98x
4. FiLM + Cross-Attn + Flow Pred + Rec + TV + FB	1.01x

For all variants, the conditioning sensitivity ratio is approximately 1.0, meaning the magnitude of the Renderer’s output does not meaningfully change when the conditioning embedding changes. In variants 1–3 (residual prediction), the absolute output magnitude converges to ≈ 0.005 —essentially near-zero. In variant 4 (flow prediction), the predicted flow field converges back to zero after initial non-zero activity from random initialization.

5.4. Diagnostic Experiments

To isolate the failure mode, we conduct several diagnostic tests:

Embedding distance test. We feed the Renderer embeddings from frames at varying temporal distances ($[t, t + 1, t + 4, t + 8, t + 16]$) and measure the output magnitude. Even with frame $t + 16$ (1-cos = 0.31, a meaningful change), the output magnitude remains unchanged from the $t + 1$ case (1-cos = 0.03). This confirms that the model is not merely limited by the weak signal in adjacent frames—it has learned to ignore the embedding entirely.

Farneback flow as pseudo-target. We compute Farneback optical flow between adjacent frames (mean magnitude 0.39 px, max 2.84 px) and use it as a supervised target. The flow head initially produces non-zero flow but converges back to zero as the reconstruction and smoothness losses dominate.

CLIP feature discrimination. We verify that CLIP embeddings *do* contain frame-to-frame information: the cosine similarity gap between same-video adjacent frames (0.976) and different videos of the same class (0.664) is 0.312—a substantial difference. The information exists, but translating it from a 512-dim global vector into spatially meaningful pixel changes proves fundamentally difficult with our architectures.

6. Discussion: Why Did It Fail?

We identify the following contributing factors, ordered from most fundamental to most specific.

6.1. Primary Cause: Frozen CLIP Embeddings Are Unsuitable for This Task

We believe this is the root cause from which all other issues derive. Several lines of evidence point specifically to the frozen pre-trained ViT embeddings—rather than the Transformer architecture or the decoupled pipeline design—as the bottleneck.

First, purely autoregressive Transformer-based video generation has been demonstrated successfully in prior work (e.g., VideoGPT, TATS, and more recent video tokenizers combined with autoregressive transformers). These models generate videos token-by-token or frame-by-frame, proving that the autoregressive Transformer paradigm is not inherently flawed for video generation. The key difference is that these models operate on *learned* discrete tokens from a VQ-VAE trained jointly for reconstruction, whereas our Flow Model operates on *frozen* CLIP embeddings that were never optimized for the rendering task.

Second, CLIP ViT-B/32 is trained with a contrastive image-text matching objective: given an image and a text description, maximize the cosine similarity of their embeddings. This objective encourages the model to extract *semantic content*—“what is in this image”—and compresses it into a compact 512-dimensional vector. What CLIP’s training objective does *not* require is preserving the kind of information needed for frame-to-frame rendering:

- **Spatial configuration:** The precise 2D coordinates of body parts, objects, and their relative positions.
- **Pose and articulation:** The joint angles of limbs, the orientation of a head, the bend of a knee.
- **Dense correspondence:** Which pixel in frame t corresponds to which pixel in frame $t + 1$.

A CLIP embedding can tell you “this is a person walking” with high confidence, but it cannot tell you “the right leg is currently forward at a 30-degree angle, positioned at pixel coordinates (120, 180).” The latter is precisely the information a Renderer needs to predict what the next frame should look like.

This explains the asymmetry in our results: the Flow Model succeeded because predicting an embedding trajectory only requires matching the statistical distribution of CLIP embeddings—a task CLIP embeddings contain sufficient information for. The Renderer failed because decoding an embedding back to a specific spatial configuration of pixels requires information that was discarded during CLIP’s encoding process.

Empirically, we confirmed this information bottleneck: adjacent frames of the same video have CLIP embeddings

with cosine similarity ≈ 0.976 ($1 - \cos \approx 0.024$). This tiny difference must encode the entire motion transformation from one frame to the next—a compression ratio far exceeding what any decoder architecture can plausibly invert.

6.2. Contributing Factor: Loss Function Misalignment

All our loss functions (L1 on residuals, L1 on reconstruction after warping) have an optimal trivial solution near the identity transformation: $\Delta I_t \approx 0$ and flow ≈ 0 . When the conditioning signal carries almost no usable information (as argued above), identity mapping becomes the rational optimum.

6.3. Contributing Factor: FiLM Cannot Express Spatial Guidance

Feature-wise Linear Modulation applies channel-wise scaling and shifting uniform across all spatial positions. Even if the embedding *did* contain spatial information, FiLM lacks the mechanism to route it to the correct spatial location. Cross-attention to CLIP patch tokens partially addresses this routing problem, but the patch tokens themselves still lack dense correspondence information due to the primary cause above.

6.4. Limitations of Our Study

We acknowledge several limitations: (1) we only explored CLIP ViT-B/32 as the embedding backbone—larger ViT variants or dense-prediction models like DINOv2 may have different properties; (2) we did not attempt joint end-to-end training of the Flow Model and Renderer, which could allow the Flow Model to learn embeddings specifically optimized for rendering; (3) we used a relatively small dataset (6.7k videos), and the collapse behavior may differ at larger scale where more diverse motion patterns provide a stronger learning signal; (4) HMDB51 videos are constrained to specific human action categories, and the learned dynamics may not generalize to more open-ended video generation.

6.5. What We Learned

Despite the negative result, this investigation yielded several lessons that may benefit future work in this area:

Embedding choice is critical. The choice of CLIP ViT-B/32 as our embedding backbone was motivated by its joint vision-language space and compact dimensionality. However, CLIP embeddings are optimized for semantic discrimination, not spatial correspondence. For a decoupled I2V pipeline to succeed, the embedding space must encode both *what* is in the scene and *where* things are spatially—properties that CLIP’s contrastive training objective does not guarantee.

Loss functions define the optimization landscape. We observed that all tested pixel-space loss functions (L1, L1 + TV, L1 + Farneback) contain a near-identity trivial solution ($\Delta I \approx 0$, flow ≈ 0). This is not specific to our architecture—any model trained with a pixel reconstruction loss on adjacent-frame pairs faces the same challenge. Loss functions that operate in a learned feature space (perceptual loss, adversarial loss) may provide a stronger signal for motion learning.

The JEPA philosophy needs a rendering pathway. JEPA’s core insight—predict in representation space, not input space—proved effective for our Flow Model (82.5% of GT motion). But the companion challenge of *decoding* those predictions back to the pixel domain is equally hard and received comparatively little attention in our implementation. The rendering problem deserves its own careful treatment rather than being treated as an architectural afterthought.

6.6. Future Work

Several promising directions emerge from this analysis.

Dense spatial embeddings. Replacing CLIP with a model trained for dense visual correspondence—such as DINOv2, Stable Diffusion’s VAE latents, or a self-supervised optical flow network—could provide embeddings that naturally encode spatial configuration. A trajectory in DINOv2 patch feature space would carry richer spatial information than a trajectory in CLIP CLS space.

Joint end-to-end training. Training the Flow Model and Renderer jointly, with gradients flowing from the rendering loss back through the Flow Model, would allow the embedding space to adapt to the needs of the Renderer. The Flow Model would no longer be constrained to match CLIP’s embedding structure and could learn features specifically optimized for frame rendering.

Iterative refinement. Rather than predicting the full frame transformation in a single forward pass, a multi-step refinement process (similar to diffusion models or recurrent refinement) could gradually build up the output from an initial near-identity estimate. This would allow the model to learn motion in stages—coarse structure first, then fine details.

Adversarial and perceptual losses. GAN-based losses or CLIP-based perceptual losses directly penalize lack of motion and unrealistic static output, potentially breaking the near-identity local minimum that trapped all of our L1-trained models.

Explicit motion field supervision. Using a pre-trained optical flow network (e.g., RAFT) instead of Farneback as a supervision target could provide a cleaner learning signal. Combined with a rendering loss, this could guide the model toward physically plausible motion while maintaining visual fidelity.

7. Conclusion

We set out to build a decoupled image-to-video generation system in which a causal Transformer predicts action-conditioned embedding trajectories and a conditioned UNet renders those trajectories to pixels. The Flow Model component was partially successful: with cosine similarity loss it learned to predict embedding trajectories preserving 82.5% of ground-truth motion variation, while MSE loss led to complete collapse. However, the Renderer failed across all four conditioning architectures tested: FiLM, cross-attention, contrastive conditioning, and optical flow prediction.

Our analysis suggests the primary cause is a fundamental information bottleneck: CLIP ViT-B/32 embeddings are trained for semantic image-text alignment and do not preserve the spatial configuration, pose, and correspondence information necessary for pixel-level frame rendering. Adjacent frames differ by less than 3% in CLIP embedding cosine distance, an information compression ratio that is implausible for any decoder architecture to invert. All other observed problems—loss function collapse to identity, FiLM’s inability to express spatial motion—can be understood as consequences of this root cause.

Returning to our original research question—whether frozen pre-trained visual priors can serve as an effective backbone for low-cost I2V generation—our results suggest a nuanced answer. The priors *do* carry useful semantic information: our Flow Model successfully learned action-conditioned embedding trajectories with 82.5% of ground-truth motion variation, and the embedding space cleanly separates different action classes (inter-class $1 - \cos \approx 0.44$ vs. intra-class ≈ 0.34). However, semantic priors alone are insufficient for the *rendering* stage, which requires fine-grained spatial information that CLIP’s training objective discards. The lesson is not that pre-trained priors are useless for video generation, but that the choice of pre-trained representation must be matched to *both* stages of a decoupled pipeline—the prediction stage needs semantic structure, while the rendering stage needs spatial correspondence.

Future directions suggested by this analysis include: (1) replacing CLIP with a dense visual representation trained for spatial correspondence, such as DINOv2 patch features, Stable Diffusion VAE latents, or self-supervised optical flow features; (2) training the Flow Model and Renderer jointly end-to-end, so the embedding space can adapt to the specific needs of rendering rather than being constrained by a frozen pre-trained encoder; (3) adopting losses that explicitly penalize lack of motion, such as adversarial or perceptual losses, to break the near-identity local minimum; (4) incorporating explicit motion representations such as optical flow or keypoint trajectories as intermediate supervision targets rather than relying solely on the embed-

ding to carry all motion information.

References

- [1] A. Blattmann, T. Dockhorn, S. Kulal, et al. “Stable Video Diffusion: Scaling Latent Video Diffusion Models to Large Datasets.” *arXiv:2311.15127*, 2023.
- [2] Alibaba. “Wan2.2: Large-Scale Image-to-Video Generation.” 2025–2026.
- [3] Y. LeCun. “A Path Towards Autonomous Machine Intelligence.” *OpenReview*, 2022.
- [4] A. Bardes, Q. Garrido, J. Ponce, et al. “Revisiting Feature Prediction for Learning Visual Representations from Video.” *arXiv:2404.08471*, 2024.
- [5] A. Radford, J. W. Kim, C. Hallacy, et al. “Learning Transferable Visual Models From Natural Language Supervision.” *ICML*, 2021.
- [6] H. Kuehne, H. Jhuang, E. Garrote, et al. “HMDB: A Large Video Database for Human Motion Recognition.” *ICCV*, 2011.
- [7] E. Perez, F. Strub, H. de Vries, et al. “FiLM: Visual Reasoning with a General Conditioning Layer.” *AAAI*, 2018.
- [8] R. Zhang, P. Isola, A. A. Efros, et al. “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric.” *CVPR*, 2018.
- [9] A. Vaswani, N. Shazeer, N. Parmar, et al. “Attention Is All You Need.” *NeurIPS*, 2017.
- [10] P. Isola, J.-Y. Zhu, T. Zhou, et al. “Image-to-Image Translation with Conditional Adversarial Networks.” *CVPR*, 2017.
- [11] O. Ronneberger, P. Fischer, T. Brox. “U-Net: Convolutional Networks for Biomedical Image Segmentation.” *MICCAI*, 2015.
- [12] J. Oh, X. Guo, H. Lee, et al. “Action-Conditional Video Prediction using Deep Networks in Atari Games.” *NIPS*, 2015.
- [13] N. Srivastava, E. Mansimov, R. Salakhutdinov. “Unsupervised Learning of Video Representations using LSTMs.” *ICML*, 2015.
- [14] C. Vondrick, H. Pirsiavash, A. Torralba. “Generating Videos with Scene Dynamics.” *NIPS*, 2016.
- [15] U. Singer, A. Polyak, T. Hayes, et al. “Make-A-Video: Text-to-Video Generation without Text-Video Data.” *arXiv:2209.14792*, 2022.
- [16] J. Ho, W. Chan, C. Saharia, et al. “Imagen Video: High Definition Video Generation with Diffusion Models.” *arXiv:2210.02303*, 2022.
- [17] M. Monfort, A. Andonian, B. Zhou, et al. “Moments in Time Dataset: One Million Videos for Event Understanding.” *TPAMI*, 2019.